

SkyHorizon Airways - Architecture Notes (Current Progress)

1. Project Goal

Build a real-world airline booking system where:

- Users can search flights
 - Users can login and book flights
 - Users can check upcoming flights
 - AI Assistant (RAG) can answer airline-related questions
 - Data comes from MySQL database instead of hardcoded Java objects
-

2. Current Architecture

Browser ↓ Controller ↓ Repository ↓ Hibernate ↓ MySQL

Response Flow:

MySQL ↓ Hibernate ↓ Repository ↓ Controller ↓ JSON ↓ Browser

3. Packages Created

com.skyhorizon.skyhorizon_airways

|—— controller | |—— HomeController

|—— model | |—— Flight | |—— FlightStatus

|—— repository | |—— FlightRepository

4. Controller

Purpose:

- Handles HTTP Requests
- Returns HTTP Responses

- Entry point for browser requests

Example:

GET /flights

Browser ↓ Controller ↓ Response

Annotation:

@RestController

Example:

@GetMapping("/flights")

Interview Point:

Controller should not contain business logic.

5. Model

Purpose:

Represents data objects.

Example:

Flight User Booking Ticket

Current Flight Entity:

id flightNumber source destination price availableSeats numberOfStops status

6. FlightStatus Enum

Purpose:

Restricts values to predefined constants.

Allowed Values:

SCHEDULED ON_TIME DELAYED BOARDING DEPARTED LANDED CANCELLED RESCHEDULED

Benefits:

- Prevents invalid values
 - Improves type safety
 - Easier maintenance
-

7. Entity

Annotation:

@Entity

Purpose:

Converts Java Class into Database Table.

Example:

Flight.java ↓ flights table

8. Primary Key

Annotations:

@Id @GeneratedValue(strategy = GenerationType.IDENTITY)

Purpose:

Unique identifier for each row.

Example:

1 2 3 4

Generated automatically by MySQL.

9. Why Long Instead of long?

Long

Default Value: null

Useful because:

- Entity may not be saved yet
- JPA can detect new entities

long

Default Value: 0

Cannot represent "not saved yet"

10. No Args Constructor

Required:

```
public Flight() { }
```

Purpose:

Hibernate uses it while creating objects from database records.

Flow:

Database Row ↓ new Flight() ↓ Set Values ↓ Return Object

11. Repository

Purpose:

Database Access Layer

Responsibilities:

```
save() findAll() findById() deleteById()
```

Annotation Concept:

@Repository (conceptually)

Current Repository:

FlightRepository

extends

JpaRepository<Flight, Long>

Meaning:

Flight = Entity/Table

Long = Primary Key Type

12. JPA

Full Form:

Java Persistence API

Purpose:

Specification for ORM

Think of it as:

Rule Book

JPA itself does not perform database operations.

13. Hibernate

Purpose:

Implementation of JPA

Think of it as:

Worker

Responsibilities:

- Convert Objects to SQL
 - Convert SQL Results to Objects
 - Create Tables
 - Update Tables
-

14. ORM

Object Relational Mapping

Purpose:

Maps Java Objects to Database Tables

Example:

Flight Object ↓ Hibernate ↓ flights Table

15. Jackson

Purpose:

Converts Java Objects into JSON

Flow:

Flight Object ↓ Jackson ↓ JSON

Example:

return flight;

↓

{ "flightNumber": "SH101" }

Process Name:

16. Current Database Flow

Flight Object ↓ @Entity ↓ JPA ↓ Hibernate ↓ MySQL Table

17. Current Flight Table

flights

Columns:

id flight_number source destination price available_seats number_of_stops status

Note:

Hibernate automatically converts:

availableSeats ↓

available_seats

(camelCase → snake_case)

18. Current Application Flow

Browser ↓ GET /flights ↓ HomeController ↓ Flight Objects ↓ Jackson ↓ JSON ↓ Browser

Future Flow:

Browser ↓ Controller ↓ Service ↓ Repository ↓ Hibernate ↓ MySQL ↓ Repository ↓ Service ↓ Controller
↓ JSON ↓ Browser

19. Important Rules Learned

Controller → Handles Requests

Service → Business Logic

Repository → Database Access

Entity → Database Table

Hibernate → SQL Generation

Jackson → JSON Generation

MySQL → Data Storage

20. Next Milestone

Application Startup ↓ CommandLineRunner ↓ flightRepository.save(...) ↓ Insert Flights ↓ MySQL

Goal:

Store first real flight records in database.